

Aynı model, sıfır rastgelelik, her token kayıtlı: maliyet artık kesin ve karşılaştırılabilir

v1'in eksiklerini kapatan yeniden koşum. Üç ajan (suffice, OpenCode 1.18.32, codex-baseline) glm-5.3-flash @ Z.ai üzerinde, **her istekte do_sample=false + temperature=0** (greedy/top-1), **her giren/çıkan token kaydedici relay ile saklanarak**, memories kapalı ve compaction 200k ile koşuldu. Maliyet üç ajan için de **API'nin döndürdüğü gerçek usage'dan** hesaplandı – v1'de codex/opencode için mümkün olmayan şey.

21 / 21 HÜCRE TAM PUAN (94/94 × 3)	188 / 188 İSTEK DETERMİNİSTİK	\$0,065-0,103 AJAN BAŞI TOPLAM (KESİN)	79 MB HAM WİRE + OTURUM KAYDI
---	---	---	--

1 · Ne değişti (v1 → v2)

KONU	V1	V2 (BEDİRHAN'IN TALEPLERİ)
Örnekleme	varsayılan (rastgele)	deterministik : do_sample=false, temperature=0, relay'de enjekte
Token kaydı	yok	tam : her istek gövdesi + her yanıt akışı + usage (79 MB)
codex/opencode maliyeti	ölçüleliyordu (tahmin)	kesin : API usage'ından, tek formül
keşif/bellek ajanı	bir turda açtı	memories=false (hepsinde)
compaction limiti	80k varsayılan	200k – daha uzun tek-bağlam görevleri
proje sayısı	6	7 (07-ARŞİV 30 dosyalık uzun-bağlam görevi eklendi)

Tüm değişiklikler yalnız lokal (home config'leri + relay); ajan koduna dokunulmadı, hiçbir şey push edilmedi.

2 · Toplam – ajan başına (kesin)

AJAN	MALİYET	SÜRE	İSTEK	TAZE TOK	CACHED TOK	CACHE %	DETERMİNİZM
codex-baseline	\$0,0653	955 s	61	159.751	810.624	83,5%	61/61
suffice	\$0,0717	1.215 s	57	129.695	869.248	87,0%	57/57
opencode	\$0,1033	2.340 s	68	189.954	985.536	83,8%	70/70

codex-b		\$0,0653
suffice		\$0,0717
opencode		\$0,1033

Okuma: opencode belirgin şekilde en pahalı (suffice'in 1,44×'i) ve en yavaş (1,9×). suffice **en az taze token** işleyip **en yüksek cache oranını** (%87) tutuyor – cache mühendisliğinin doğrudan ölçümü. codex-baseline bu turda toplamda en ucuz: yalın harness kısa görevlerde çevik ve bu koşuda görevler patlamaya yol açacak kadar uzamadı; yine de suffice'ten **%23 daha çok taze token** yakıyor.

3 • H cre h cre – skor • maliyet • s re

PROJE	SUFFICE (\$ / S)		OPENCODE (\$ / S)		CODEX-B (\$ / S)	
01-kanbagi	0,0087	119	0,0089	94	0,0075	74
02-sahra	0,0057	101	0,0176	225	0,0136	196
03-cekirdek	0,0043	69	0,0065	63	0,0045	56
04-mutabakat	0,0120	247	0,0293	959	0,0074	98
05-destek	0,0199	312	0,0185	259	0,0152	189
06-otograd	0,0150	263	0,0171	682	0,0117	248
07-arsiv	0,0061	104	0,0054	58	0,0054	94
Toplam	0,0717	1.215	0,1033	2.340	0,0653	955

Skor: 21 h crenin t m  tam puan (01:15/15 · 02:11/11 · 03:12/12 · 04:14/14 · 05:16/16 · 06:17/17 · 07:9/9). Vurgu: satır bazında en d   k maliyet.

opencode'un iki patlaması: 04-mutabakat **959 s / \$0,0293** ve 06-otograd **682 s** – v1'deki aynı davranış determinizm altında da tekrarlandı. Deneme-yanılma d ng s ne girdi inde ba lamı verimli y netemiyor. suffice ve codex-baseline'da hi bir h crede b yle bir patlama yok.

02-sahra  arpıcı  rnek: aynı model, aynı deterministik ayar, aynı g rev – suffice **\$0,0057 / 101 s**, opencode **\$0,0176 / 225 s**. suffice 3,1× daha ucuz, 2,2× daha hızlı; farkın kayna ı y ksek cache oranı ve daha az istek.

4 • Determinizm ve tam kayıt

Determinizm

188 iste in **tamamına** relay tarafından `do_sample=false` `temperature=0` enjekte edildi (suffice 57/57, opencode 70/70, codex 61/61). Model her adımda en y ksek olasılıklı token'ı se ti;  ansa ba lı maliyet dalgalanması elendi.

D r st sınır: sunucu tarafı greedy bile donanım/paralellik nedeniyle nadiren bit-d zeyi oynayabilir; istemci tarafındaki t m rastgelelik kaynakları kapatıldı.

Tam kayıt (79 MB)

Her h crede `runs-v2/<proje>--<ajan>/` altında: **wire/** – her iste in tam g vdesi (modele giren her token) + her yanıt akışı ( retilen her token + usage), **index.jsonl** – istek ba ı  zet, ajanın **oturum d k m ** (suffice analytics+sessions, codex sessions, opencode storage) ve **cost.json / score.txt / meta.json**.

Hi bir filtreleme yok – Bedirhan'ın "her bilgi kısıntısı lazım" talebi birebir: ham veri, oldu u gibi.

5 • Bulgular

#	BULGU
B1	Doğruluk yine ayırtırmıyor (21/21 tam puan); üç ajan da deterministik modda 94 testi geçiyor.
B2	opencode en pahalı ve en yavaş – suffice'in 1,44× parası, 1,9× süresi; iki hücrede ciddi patlama.
B3	suffice en verimli token profili : en az taze token, %87 cache – cache/harness mühendisliğinin ölçülen karşılığı.
B4	codex-baseline bu turda toplam en ucuz ama en çok taze token'ı da o yakıyor; avantajı görevlerin bu kez patlamaya yol açmaması. Uzun/karmaşık görev arttıkça makasın suffice lehine açılması beklenir (bkz. 02-sahra).
B5	Maliyet artık kesin ve tekrar-üretilebilir : deterministik + tam wire kaydı sayesinde her rakam ham veriden yeniden doğrulanabilir. v1'in en büyük eksiği kapandı.

Fiyat: Z.ai glm-5.3-flash – \$0,15 taze / \$0,03 cached / \$0,50 çıktı (1M başına). Üretim: Claude (Fable 5). Ham veri: ~/agent-bench/runs-v2 · report/v2-data.json. Testler koşum öncesi referans çözümlerle 94/94 doğrulandı.